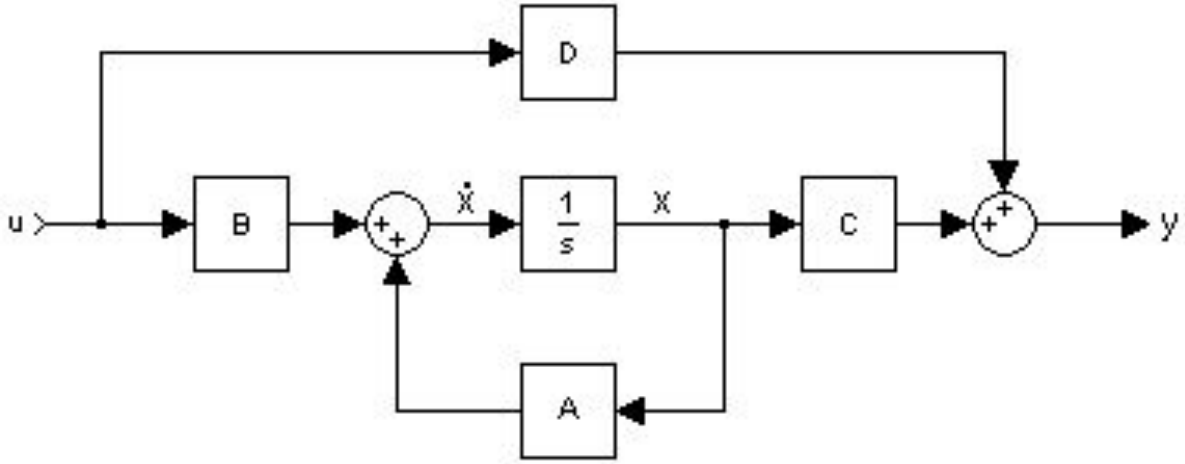
Mamba
Linear-Time Sequence Modeling with Selective State Spaces

발표자 : 정강민

State Space



State equation

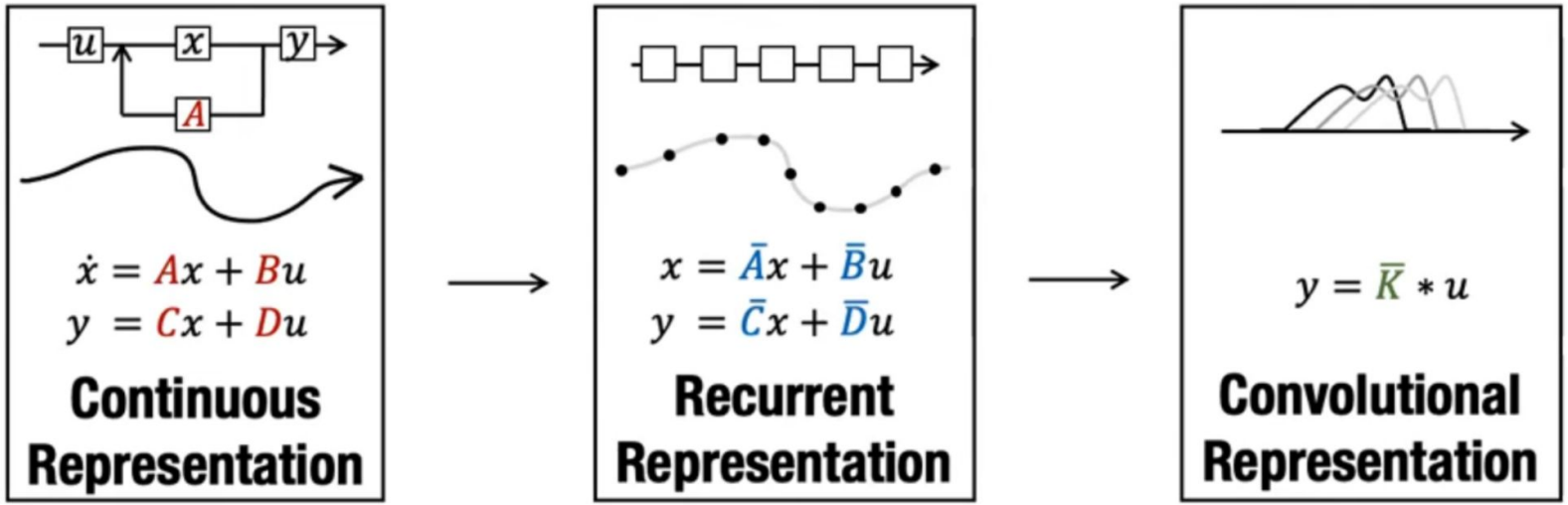
$$\dot{\boldsymbol{X}} = \boldsymbol{AX} + \boldsymbol{Bu}$$

Output equation

$$\boldsymbol{Y} = \boldsymbol{CX} + \boldsymbol{Du}$$

Where $A \in \mathbb{R}^{n \times n}$: System matrix
$B \in \mathbb{R}^{n}$: Input matrix
$C \in \mathbb{R}^{n}$: Output matrix
$D \in \mathbb{R}^{n}$: Feedforward matrix
$u \in \mathbb{R}^{n}$: Input or control vector

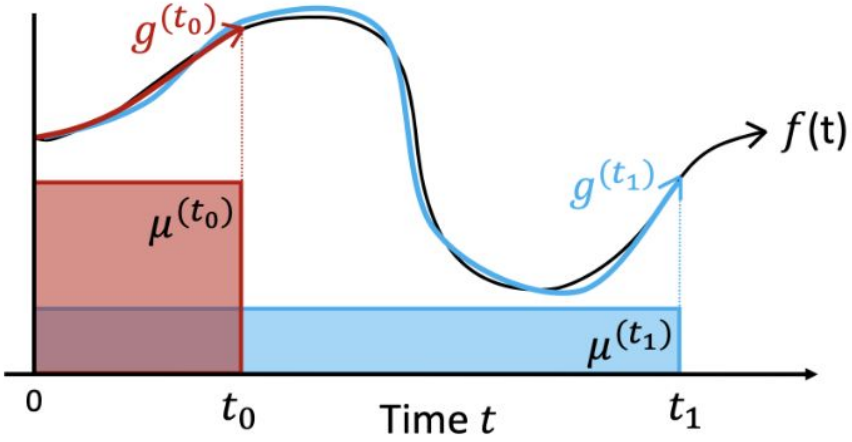
Sequence Model



SSM, State Space Models

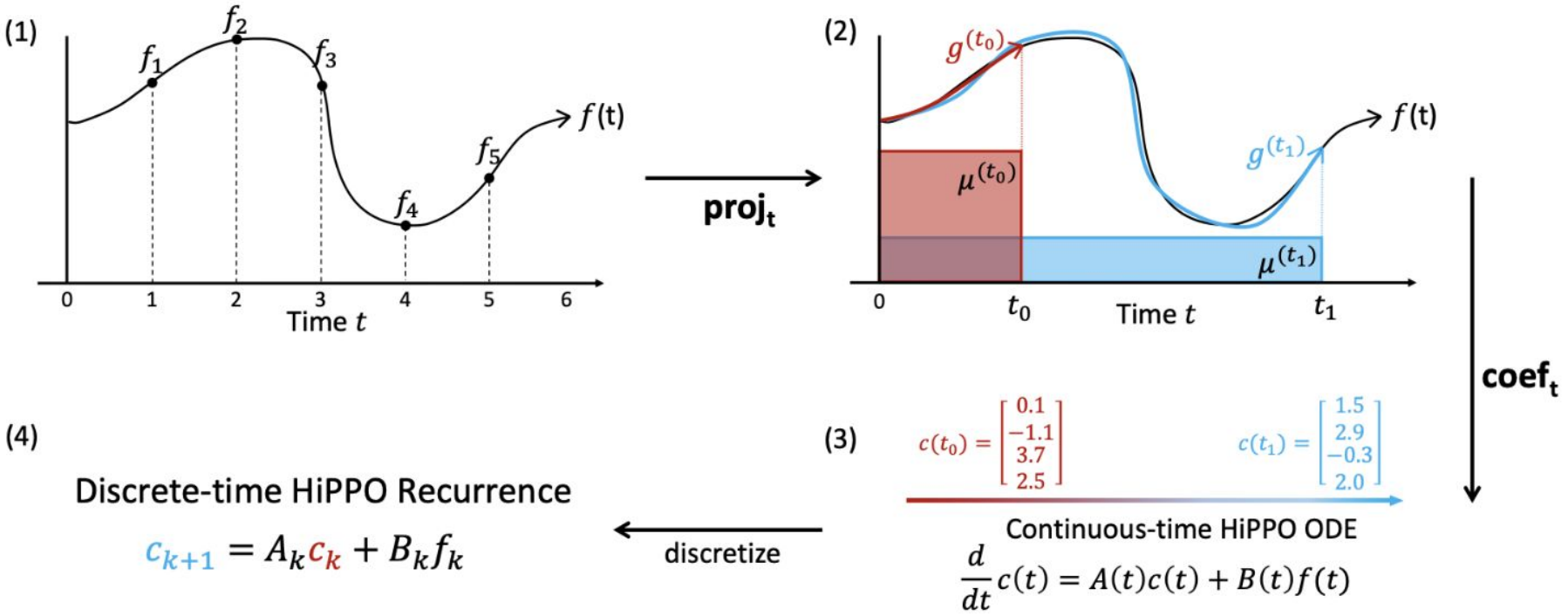
$$x'(t) = \boldsymbol{A}x(t) + \boldsymbol{B}u(t)$$
$$y(t) = \boldsymbol{C}x(t) + \boldsymbol{D}u(t)$$

HiPPO



$$c(t_0) = \begin{bmatrix} 0.1 \\ -1.1 \\ 3.7 \\ 2.5 \end{bmatrix} \qquad c(t_1) = \begin{bmatrix} 1.5 \\ 2.9 \\ -0.3 \\ 2.0 \end{bmatrix}$$

HiPPO


(1)
f_1
f_2
f_3
f_4
f_5
$f(t)$
0 1 2 3 4 5 6
Time t
proj_t
(2)
$g^{(t_0)}$
$g^{(t_1)}$
$\mu^{(t_0)}$
$\mu^{(t_1)}$
$f(t)$
0 t_0 Time t t_1
coef_t
(3)
$c(t_0) = \begin{bmatrix} 0.1 \\ -1.1 \\ 3.7 \\ 2.5 \end{bmatrix}$
$c(t_1) = \begin{bmatrix} 1.5 \\ 2.9 \\ -0.3 \\ 2.0 \end{bmatrix}$
Continuous-time HiPPO ODE
$\frac{d}{dt}c(t) = A(t)c(t) + B(t)f(t)$
discretize
(4)
Discrete-time HiPPO Recurrence
$c_{k+1} = A_k c_k + B_k f_k$

Discretization

$$h'(t) = Ah(t) + Bx(t)$$
$$y(t) = Ch(t)$$

$$h_t = \overline{\boldsymbol{A}} h_{t-1} + \overline{\boldsymbol{B}} x_t$$
$$y_t = \boldsymbol{C} h_t$$

$$\overline{\boldsymbol{A}} = \exp(\Delta \boldsymbol{A}) \quad \overline{\boldsymbol{B}} = (\Delta \boldsymbol{A})^{-1}(\exp(\Delta \boldsymbol{A}) - \boldsymbol{I}) \cdot \Delta \boldsymbol{B}$$

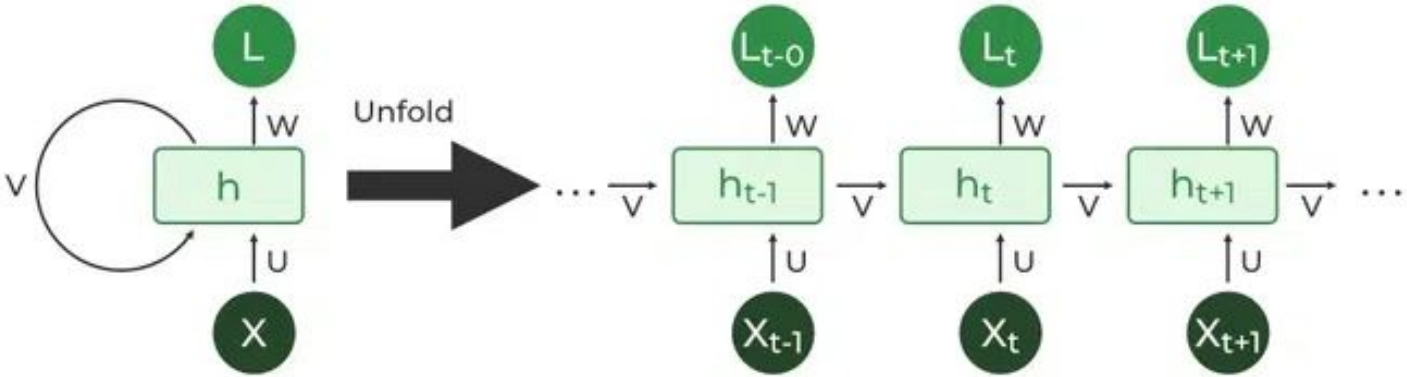
Recurrent computation

시스템의 초기 상태가 0이라고 가정

$$h_t = \overline{\boldsymbol{A}} h_{t-1} + \overline{\boldsymbol{B}} x_t$$
$$y_t = \boldsymbol{C} h_t$$

$$h_0 = \bar{B} x_0 \qquad h_1 = \bar{A} h_0 + \bar{B} x_1 \qquad h_2 = \bar{A} h_1 + \bar{B} x_2$$
$$y_0 = C h_0 \qquad y_1 = C h_1 \qquad y_2 = C h_2$$

Recurrent computation: the problem



Convolutional computation

$$h_0 = \bar{B}x_0$$
$$y_0 = Ch_0 = C\bar{B}x_0$$

$$h_t = \overline{\boldsymbol{A}}h_{t-1} + \overline{\boldsymbol{B}}x_t$$
$$y_t = \boldsymbol{C}h_t$$

$$h_1 = \bar{A}h_0 + \bar{B}x_1 = \bar{A}\bar{B}x_0 + \bar{B}x_1$$
$$y_1 = Ch_1 = C\Big(\bar{A}\bar{B}x_0 + \bar{B}x_1\Big) = C\bar{A}\bar{B}x_0 + C\bar{B}x_1$$

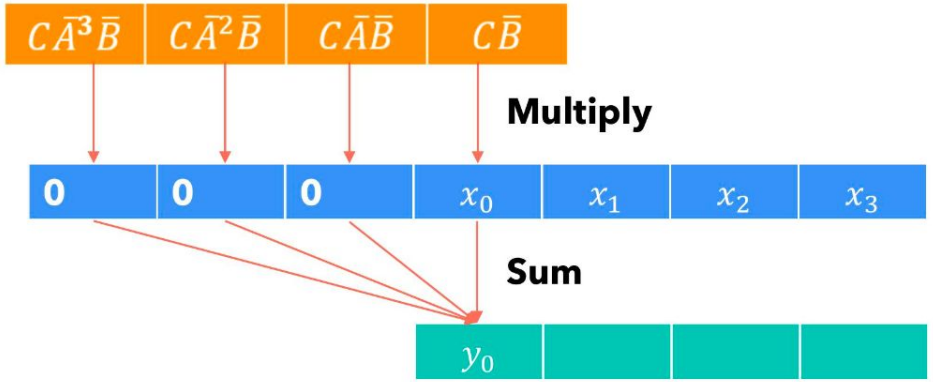
$$h_2 = \bar{A}h_1 + \bar{B}x_2 = \bar{A}\Big(\bar{A}\bar{B}x_0 + \bar{B}x_1\Big) + \bar{B}x_2 = \bar{A}^2\bar{B}x_0 + \bar{A}\bar{B}x_1 + \bar{B}x_2$$
$$y_2 = Ch_2 = C\Big(\bar{A}^2\bar{B}x_0 + \bar{A}\bar{B}x_1 + \bar{B}x_2\Big) = C\bar{A}^2\bar{B}x_0 + C\bar{A}\bar{B}x_1 + C\bar{B}x_2$$

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$\overline{\boldsymbol{K}} = (C\overline{\boldsymbol{B}}, C\overline{\boldsymbol{AB}}, \ldots, C\overline{\boldsymbol{A}}^{\boldsymbol{k}}\overline{\boldsymbol{B}}, \ldots) \quad (3a)$$

$$y = x * \overline{\boldsymbol{K}} \quad (3b)$$



Kernel

Input

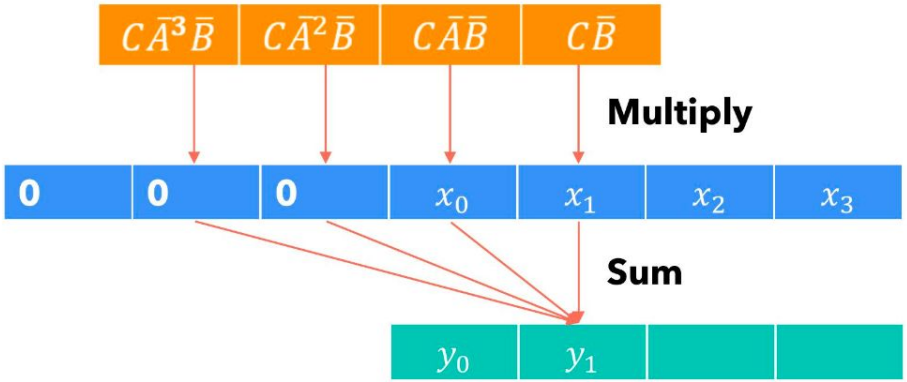
Output

$$y_0 = C\bar{B}x_0$$

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$\overline{\boldsymbol{K}} = (C\overline{\boldsymbol{B}}, C\overline{\boldsymbol{AB}}, \ldots, C\overline{\boldsymbol{A}}^{\boldsymbol{k}}\overline{\boldsymbol{B}}, \ldots) \quad (3a)$$

$$y = x * \overline{\boldsymbol{K}} \quad (3b)$$



Kernel

Input

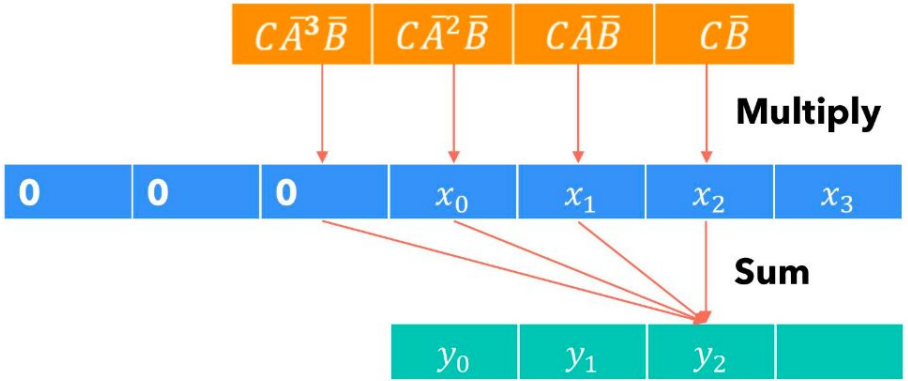
Output

$$y_1 = C\bar{A}\bar{B}x_0 + C\bar{B}x_1$$

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$\overline{\boldsymbol{K}} = (C\overline{\boldsymbol{B}}, C\overline{\boldsymbol{AB}}, \ldots, C\overline{\boldsymbol{A}}^{\boldsymbol{k}}\overline{\boldsymbol{B}}, \ldots) \quad (3a)$$

$$y = x * \overline{\boldsymbol{K}} \quad (3b)$$

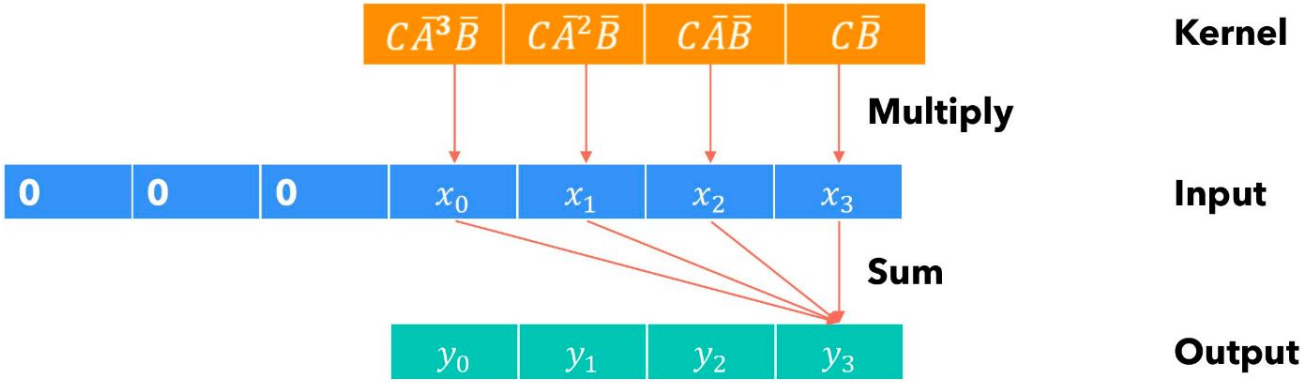


$$y_2 = C\bar{A}^2\bar{B}x_0 + C\bar{A}\bar{B}x_1 + C\bar{B}x_2$$

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$\overline{\boldsymbol{K}} = (C\overline{\boldsymbol{B}}, C\overline{\boldsymbol{AB}}, \ldots, C\overline{\boldsymbol{A}}^{\boldsymbol{k}}\overline{\boldsymbol{B}}, \ldots) \quad (3a)$$

$$y = x * \overline{\boldsymbol{K}} \quad (3b)$$



$$y_3 = C\bar{A}^3\bar{B}x_0 + C\bar{A}^2\bar{B}x_1 + C\bar{A}\bar{B}x_2 + C\bar{B}x_3$$

Convolutional & Recurrent

Convolutional computation

- 병렬 처리 가능: 모든 입력 시퀀스 토큰에 대해 훈련하기에 적합
- 출력은 커널과 입력에만 의존하며, 이전 출력에 의존하지 않음
- 커널을 구현하는 과정이 계산 및 메모리 측면에서 비용이 많이 듦

Recurrent computation

- 한 번에 한 토큰씩 일정한 계산과 메모리를 사용하여 처리
- 병렬 처리에 어려움이 있으며 기울기 소실/폭발 문제가 발생할 수 있음
- 순차적 의존성으로 인해 대규모 데이터셋 또는 긴 시퀀스 처리에 비효율적

Algorithm 1 SSM (S4)

Input: $x : (\mathtt{B}, \mathtt{L}, \mathtt{D})$
Output: $y : (\mathtt{B}, \mathtt{L}, \mathtt{D})$

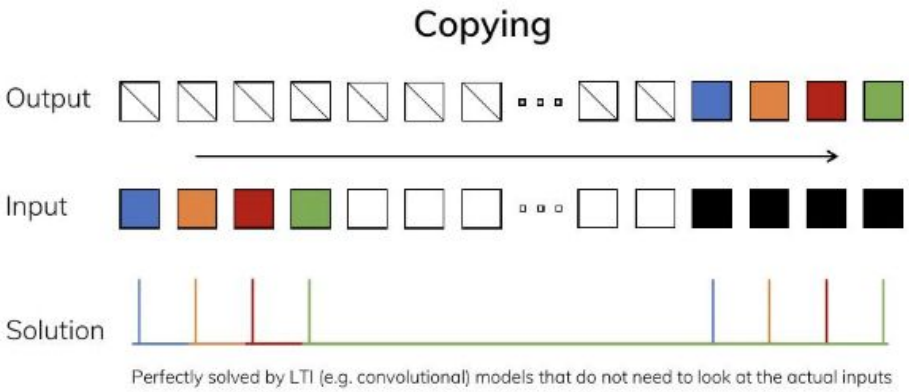
$\boldsymbol{A} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
$\boldsymbol{C} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
$\Delta : (\mathtt{D}) \leftarrow \tau_\Delta(\text{Parameter})$
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathtt{D}, \mathtt{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
return y

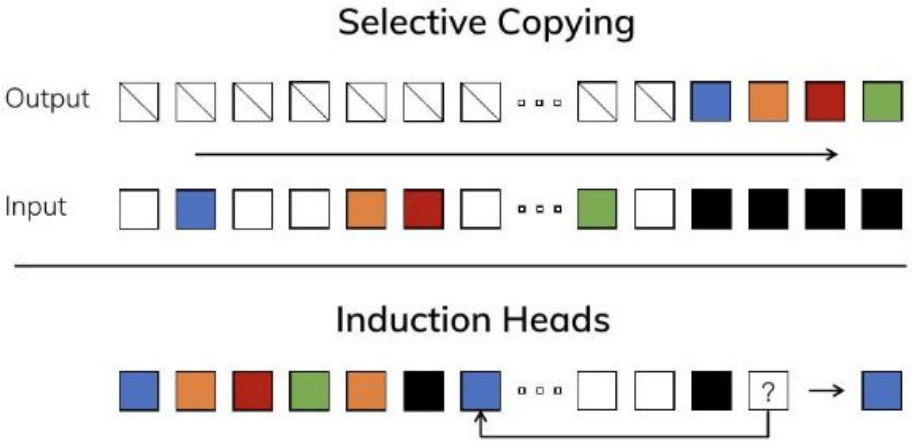
Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathtt{B}, \mathtt{L}, \mathtt{D})$
Output: $y : (\mathtt{B}, \mathtt{L}, \mathtt{D})$

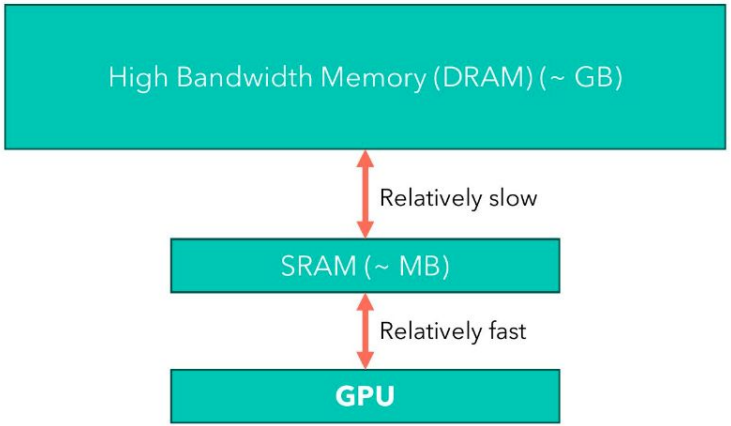
$\boldsymbol{A} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (\mathtt{B}, \mathtt{L}, \mathtt{N}) \leftarrow s_B(x)$
$\boldsymbol{C} : (\mathtt{B}, \mathtt{L}, \mathtt{N}) \leftarrow s_C(x)$
$\Delta : (\mathtt{B}, \mathtt{L}, \mathtt{D}) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathtt{B}, \mathtt{L}, \mathtt{D}, \mathtt{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
return y

Improving SSMs with Selection





The GPU memory hierarchy



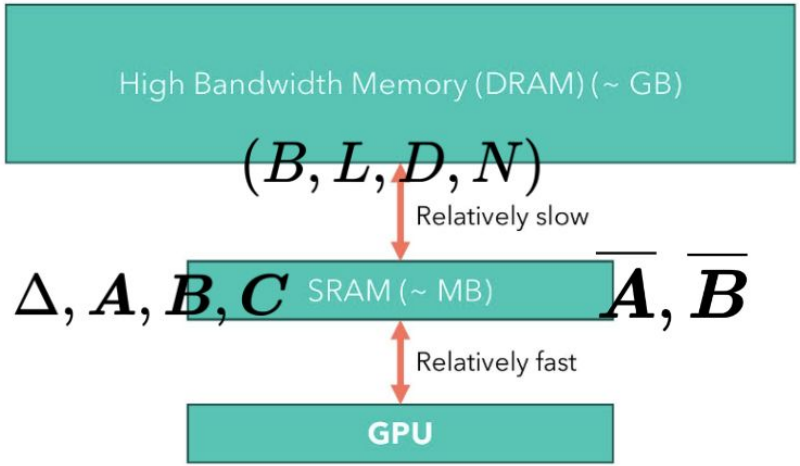
Because the GPU is not so fast at moving tensor around, but it's super fast at computing operations, sometimes, the problem in our algorithm may not be the number of computations we do, but how many tensors we move around in the different memory hierarchies. In this case, we say that the operation is IO-bound.

NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS (SXM4 AND PCIE FORM FACTORS)

	A100 40GB PCIe	A100 80GB PCIe	A100 40GB SXM	A100 80GB SXM
FP64	9.7 TFLOPS			
FP64 Tensor Core	19.5 TFLOPS			
FP32	19.5 TFLOPS			
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*			
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*			
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*			
INT8 Tensor Core	624 TOPS | 1248 TOPS*			
GPU Memory	40GB HBM2	80GB HBM2e	40GB HBM2	80GB HBM2e
GPU Memory Bandwidth	1,555GB/s	1,935GB/s	1,555GB/s	2,039GB/s
Max Thermal Design Power (TDP)	250W	300W	400W	400W
Multi-Instance GPU	Up to 7 MIGs @ 5GB	Up to 7 MIGs @ 10GB	Up to 7 MIGs @ 5GB	Up to 7 MIGs @ 10GB
Form Factor	PCIe		SXM	
Interconnect	NVIDIA® NVLink® Bridge for 2 GPUs: 600GB/s ** PCIe Gen4: 64GB/s		NVLink: 600GB/s PCIe Gen4: 64GB/s	
Server Options	Partner and NVIDIA-Certified Systems™ with 1-8 GPUs		NVIDIA HGX™ A100-Partner and NVIDIA-Certified Systems with 4,8, or 16 GPUs NVIDIA DGX™ A100 with 8 GPUs	

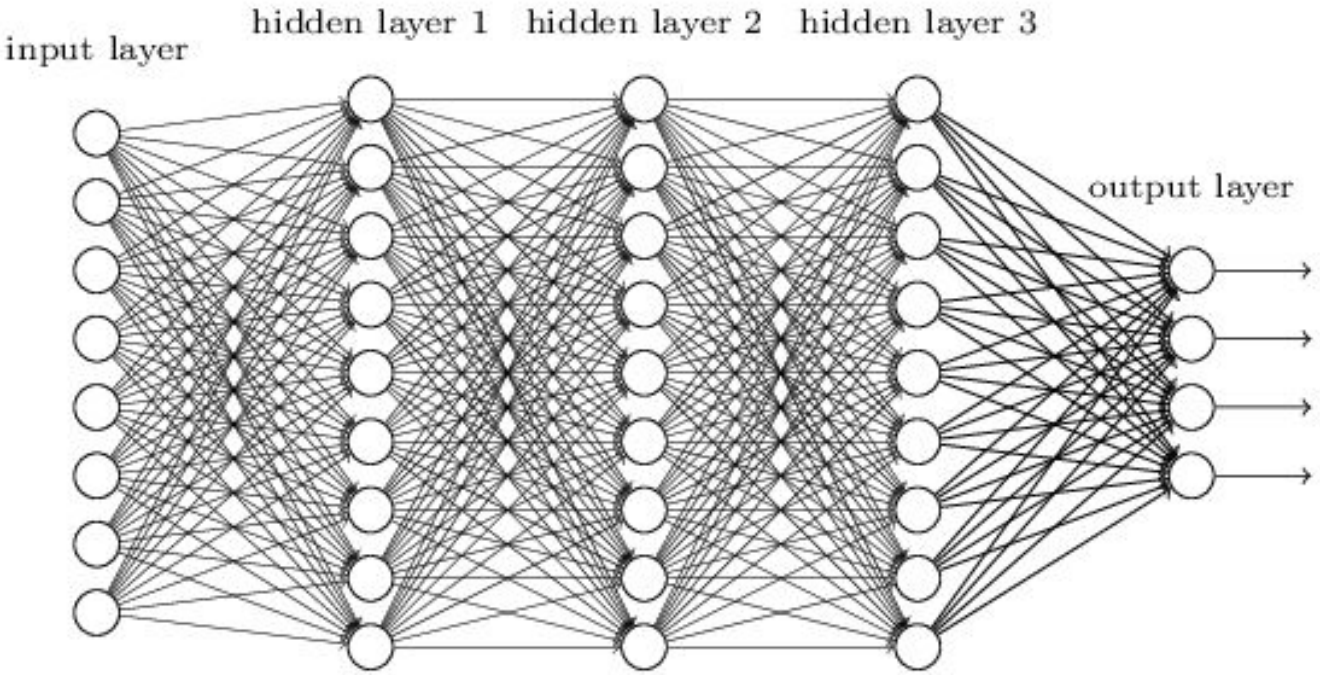
* With sparsity

** SXM4 GPUs via HGX A100 server boards; PCIe GPUs via NVLink Bridge for up to two GPUs

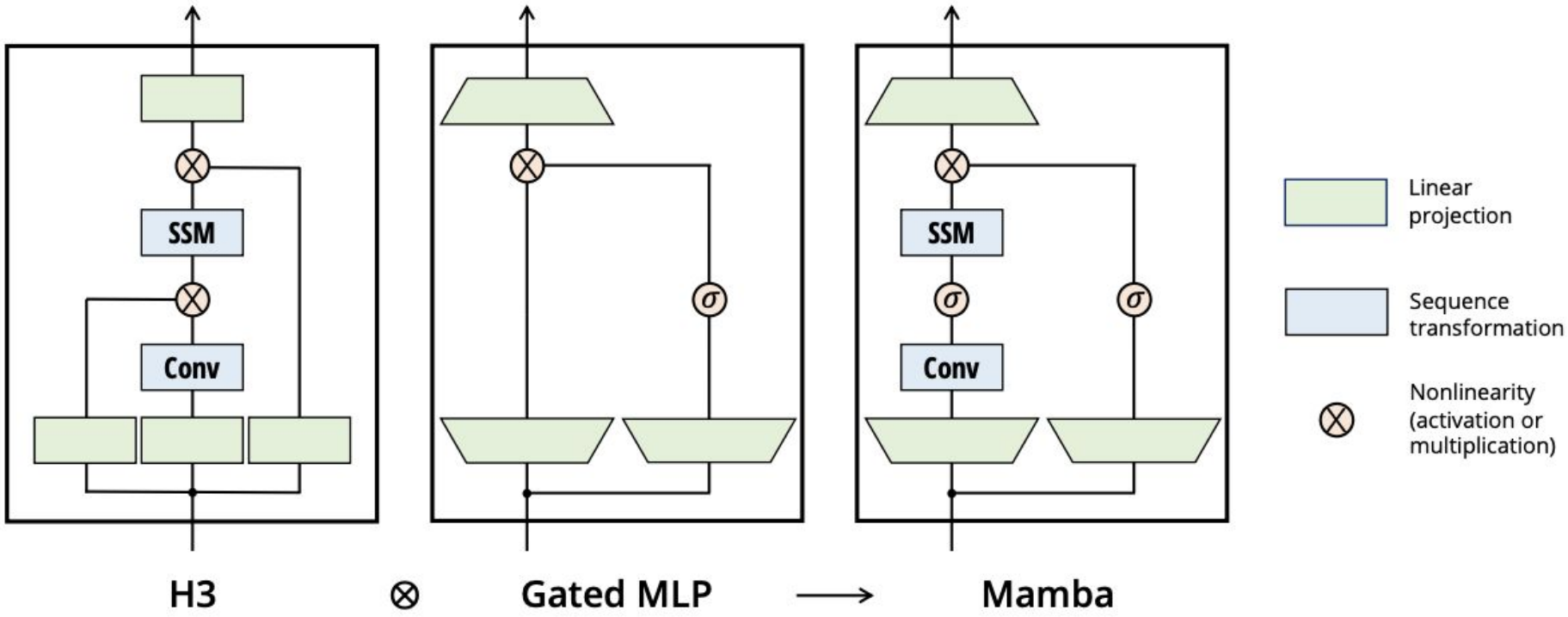


1. We read in $O(BLD + DN)$ bytes of memory $(\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C})$ from slow HBM to fast SRAM.
2. We discretize to produce $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}$ of size (B, L, D, N) in SRAM.
3. We perform a parallel associative scan, yielding intermediate states of size (B, L, D, N) in SRAM.
4. We multiply and sum with $\boldsymbol{C}$, producing outputs of size (B, L, D) and write it to HBM.

Mamba: Recomputation



Mamba block


SSM
Conv
SSM
Conv
H3
Gated MLP
Mamba
Linear projection
Sequence transformation
Nonlinearity (activation or multiplication)

Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	**97.0**
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	**99.7**
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	**99.8**

Table 1: (**Selective Copying**.) Accuracy for combinations of architectures and inner sequence layers.

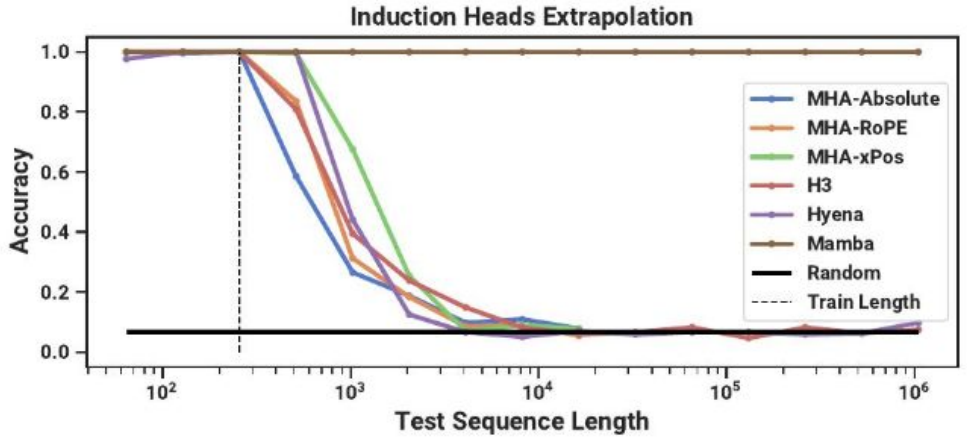
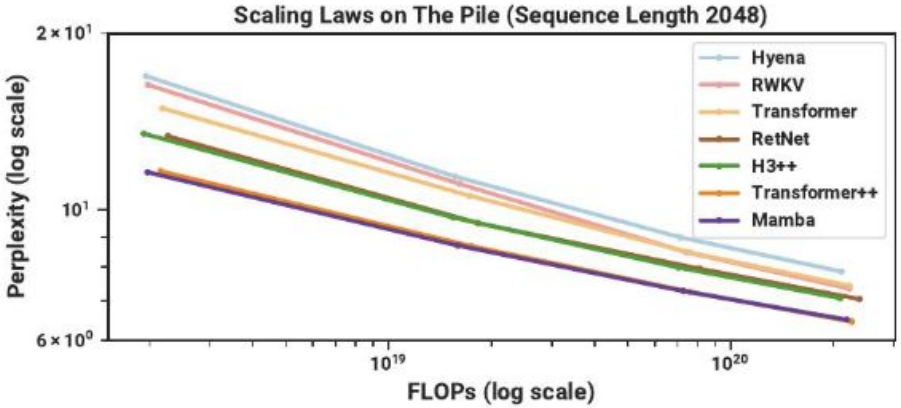
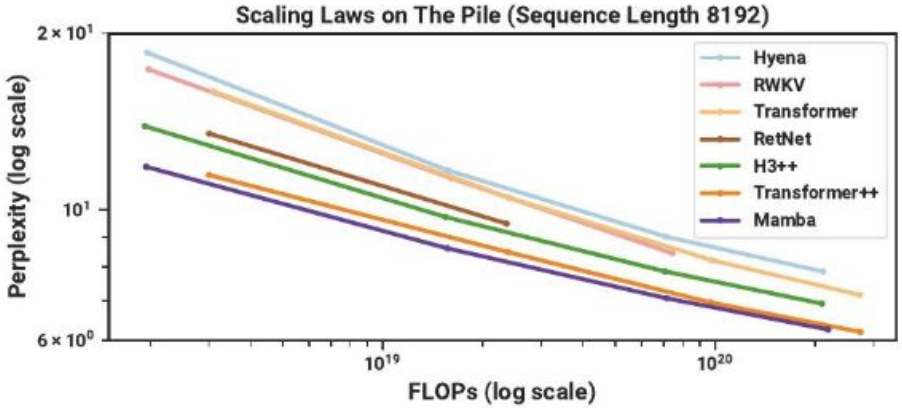


Table 2: (**Induction Heads**.) Models are trained on sequence length $2^8 = 256$, and tested on increasing sequence lengths of $2^6 = 64$ up to $2^{20} = 1048576$. Full numbers in Table 11.


Scaling Laws on The Pile (Sequence Length 2048)
Perplexity (log scale)
FLOPs (log scale)
Hyena
RWKV
Transformer
RetNet
H3++
Transformer++
Mamba


Scaling Laws on The Pile (Sequence Length 8192)
Perplexity (log scale)
FLOPs (log scale)
Hyena
RWKV
Transformer
RetNet
H3++
Transformer++
Mamba